Stack Using Queue [push 10, 20, 30 pop( )]

Step	Operation	q1	q2	O/P
1	Start	[ ]	[ ]	
2	push 10 → add to q2	[ ]	[10]	
3	Swap	[10]	[ ]	
4	push 20 → add to q2	[10]	[20]	
5	move q1 → q2	[ ]	[20, 10]	
6	Swap	[20, 10]	[ ]	
7	push 30 → add to q2	[20, 10]	[30]	
8	move q1 − q2	[10]	[30, 20]	
9	move q1 → q2	[ ]	[30, 20, 10]	
10	Swap	[30, 20, 10]	[ ]	
11	pop	[20, 10]	[ ]	30

Queue Using Stack enqueue 10, 20, 30 dequeue ( )

Step	Operation	S1	S2	O/P
1	Start	[ ]	[ ]	
2	enqueue (10)	[10]	[ ]	
3	enqueue (20)	[10, 20]	[ ]	
4	enqueue (30)	[10, 20, 30]	[ ]	
5	dequeue → move S1 → S2	[10, 20]	[30]	
6	dequeue → move S1 → S2	[10]	[30, 20]	
7	dequeue → move S1 → S2	[ ]	[30, 20, 10]	
8	pop from S2	[ ]	[30, 20]	10

Sliding Window Problems :→ [Competitive Coding]

int[ ] arr = {1, 3, -1, -3, 5, 3, 6, 7}; k = 3

	Max
[1, 3, -1,] -3, 5, 3, 6, 7	3
1, [3, -1, -3,] 5, 3, 6, 7	3
1, 3, [-1, -3, 5,] 3, 6, 7	5
1, 3, -1, [-3, 5, 3,] 6, 7	5
1, 3, -1, -3, [5, 3, 6,] 7	6
1, 3, -1, -3, 5, [3, 6, 7]	7

for (1 to n
for (1 to k
O(n*k)

* Find the max in each window size k

Brute Force

* Sliding Window Max Using Array Deque :→

array = {1, 3, -1, -3, 5, 3, 6, 7}
(indexes: 0, 1, 2, 3, 4, 5, 6, 7)

i	arr[i]	Deque (indexes)	Values in Deque	Output
0	1	[0]	× [1]	
1	3	[1]	[3]	
2	−1	[1, 2]	[3, −1]	→ 3
3	−3	[1, 2, 3]	⇒ [3, −1, −3]	→ 3
4	5	[4]	[5]	→ 5
5	3	[4, 5]	[5, 3]	→ 5
6	6	[6]	[6]	→ 6
7	7	[7]	[7]	→ 7

Agenda for tomorrow : 14th March

Binary Tree → Theory + Traversals (DFS + BFS)

Implementation using class (OOPs Concepts)

* Understand the recursive approach.

* Interview Questions

* Binary Search Tree

* Operations → Insert / Search / Delete (imp) ******

* Limitations & Drawbacks of BST